

5. Write a program for simple RSA algorithm to encrypt and decrypt the data.

```
import java.math.BigInteger;
import java.nio.charset.StandardCharsets;
import java.util.Random;
import java.util.Scanner;

public class RSABytes {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Random rnd = new Random(System.currentTimeMillis());

        System.out.println("Enter two prime numbers (or enter 0 0 to auto-generate suitable primes):");
        BigInteger p = sc.nextBigInteger();
        BigInteger q = sc.nextBigInteger();

        // Auto-generate primes if user enters 0 0 or if provided primes are too small
        if (p.equals(BigInteger.ZERO) && q.equals(BigInteger.ZERO)) {
            int bitLength = 16; // start reasonably small but will increase if necessary
            do {
                p = BigInteger.probablePrime(bitLength, rnd);
                q = BigInteger.probablePrime(bitLength, rnd);
                bitLength++;
            } while (p.multiply(q).compareTo(BigInteger.valueOf(256)) <= 0);
            System.out.println("Auto-generated primes:");
            System.out.println("p = " + p);
            System.out.println("q = " + q);
        }

        BigInteger n = p.multiply(q);
        BigInteger phi = (p.subtract(BigInteger.ONE)).multiply(q.subtract(BigInteger.ONE));

        // Ensure modulus large enough to encrypt bytes (0..255)
        if (n.compareTo(BigInteger.valueOf(256)) <= 0) {
            System.out.println("Error: modulus n = p*q is too small for byte-wise encryption. Use larger primes.");
            sc.close();
            return;
        }

        // Choose a public exponent e (prefer 65537, fallback to small odd values)
        BigInteger e = BigInteger.valueOf(65537);
        if (!phi.gcd(e).equals(BigInteger.ONE)) {
```

```

    e = BigInteger.valueOf(3);
    while (!phi.gcd(e).equals(BigInteger.ONE)) {
        e = e.add(BigInteger.TWO);
    }
}

// Compute private exponent d
BigInteger d;
try {
    d = e.modInverse(phi);
} catch (ArithmeticException ex) {
    // fallback: choose another e (shouldn't normally happen because we checked gcd
    // above)
    e = BigInteger.valueOf(3);
    while (!phi.gcd(e).equals(BigInteger.ONE)) e = e.add(BigInteger.TWO);
    d = e.modInverse(phi);
}

System.out.println("\nPublic key (e): " + e);
System.out.println("Private key (d): " + d);
System.out.println("Modulus (n): " + n);

sc.nextLine(); // consume leftover newline
System.out.println("\nEnter the message (whole line allowed):");
String plaintext = sc.nextLine();

// Convert to UTF-8 bytes (handles spaces and multibyte chars)
byte[] plainBytes = plaintext.getBytes(StandardCharsets.UTF_8);

// Encrypt per byte (each byte becomes an integer 0..255 then encrypted)
BigInteger[] cipher = new BigInteger[plainBytes.length];
for (int i = 0; i < plainBytes.length; i++) {
    int unsigned = plainBytes[i] & 0xFF; // 0..255
    BigInteger m = BigInteger.valueOf(unsigned);
    cipher[i] = m.modPow(e, n);
}

System.out.println("\nCiphertext (space-separated integers):");
StringBuilder cb = new StringBuilder();
for (BigInteger c : cipher) {
    cb.append(c.toString()).append(" ");
}
System.out.println(cb.toString().trim());

// Decrypt and rebuild bytes
byte[] decryptedBytes = new byte[cipher.length];

```

```

for (int i = 0; i < cipher.length; i++) {
    BigInteger m = cipher[i].modPow(d, n);
    int val = m.intValue(); // should be 0..255
    decryptedBytes[i] = (byte) (val & 0xFF);
}

String decrypted = new String(decryptedBytes, StandardCharsets.UTF_8);
System.out.println("\nDecrypted message: " + decrypted);

sc.close();
}
}

```

OUTPUT:

Enter two prime numbers (or enter 0 0 to auto-generate suitable primes):

61 53

Public key (e): 65537

Private key (d): 2753

Modulus (n): 3233

Enter the message (whole line allowed):

HELLO WORLD

Ciphertext (space-separated integers):

3000 28 2726 2726 1307 1992 604 1307 1859 2726 1759

Decrypted message: HELLO WORLD